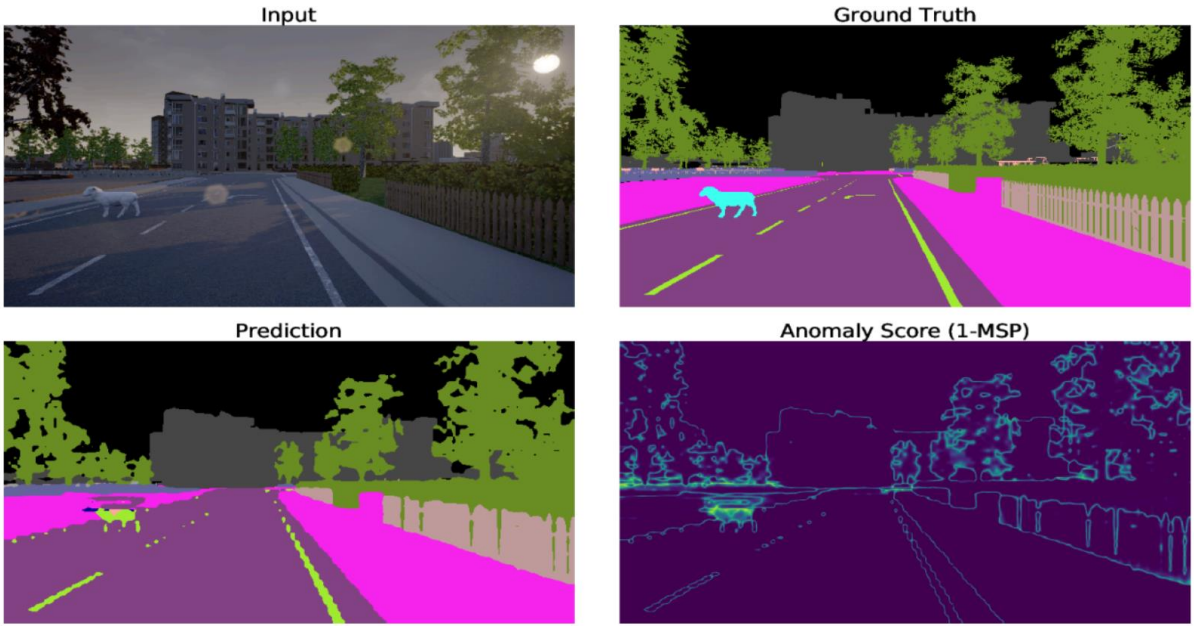
Comprehensive Road Scene Understanding for Autonomous Driving

TA: Alessandro Marinai -alessandro.marinai@polito.it

Davide Sferrazza - davide.sferrazza@polito.it

Stephany Ortuno - stephany.ortuno@polito.it



Modern computer vision has seen remarkable progress through deep neural networks, particularly in tasks such as **semantic, instance, and panoptic segmentation**. Semantic segmentation assigns a class label to every pixel in an image, instance segmentation distinguishes between different object instances of the same class, and panoptic segmentation unifies both approaches by providing a comprehensive scene understanding at both the class and instance levels. These advances have enabled systems to achieve highly accurate perception in structured and well-defined environments.

However, existing deep networks, when deployed in open-world settings, perform poorly on **unknown, anomalous, or out-of-distribution (OoD) objects** that were not present during training. This limitation poses a significant challenge, as real-world environments are inherently unpredictable and may contain rare or unseen objects. Detecting OoD objects therefore becomes critical for safety-sensitive applications such as autonomous driving, as well as for broader areas of computer vision including continual learning and open-world recognition.

In this project, the objective is first to understand the key concepts behind semantic, instance, and panoptic segmentation, and to explore their differences and practical implications. Building on this foundation, the objective is to design, train, and evaluate an **anomaly segmentation model** capable of identifying and segmenting anomalous regions in road scenes, enabling more robust perception in real-world conditions.

Starting code repository: Github Repository

Project steps

1. **Understand semantic segmentation task**
   Study a very simple and efficient model for real-time semantic segmentation ERFNet[10].

2. **Understand instance and panoptic segmentation task**
   Read the paper that defined the panoptic segmentation [11] task where also instance segmentation is referred.

3. **Study the mask architecture literature for semantic, instance and panoptic segmentation task**
   Study the core concepts behind mask architectures and the first MaskFormer model [3], understand the key changes in Mask2former, its evolution [4] (not mandatory but important) and understand the use of DINOv2 [5] in the EoMT model [6].

4. **Compare the two trained versions of EoMT on semantic, instance and panoptic**

We provide 2 different trained versions of EoMT: one trained on Cityscapes for the class of semantic segmentation and one on COCO dataset trained on the class of Panoptic Segmentation in the drive folder.

- Download the validation set of Cityscapes
- Visualize the predictions of both models on some sample images from the validation set downloaded and understand the differences of the class space and of the task they are trained on (include an image of the two predictions on the report). Remember that the two tasks are different: for the COCO-trained model visualization you should visualize a panoptic prediction, while for the Cityscapes-trained model you should visualize just a semantic segmentation prediction.
- Evaluate quantitatively on the task of semantic segmentation on the whole validation set of cityscapes both the models keeping in mind that the classes predicted are different. **N.B.** This comparison is non-trivial due to the mismatch between COCO and Cityscapes class spaces. You should define a consistent evaluation strategy that allows fair comparison across both models for the semantic segmentation task. It is essential to use the *same evaluation pipeline* for both models to ensure a fair and meaningful assessment of their performance.

5. **Fine tune the COCO-trained version:**
   Since the COCO-trained version is not specifically trained on road scenes, differences in performance are expected when evaluating it on driving-related data. However, COCO provides broader visual prior knowledge due to its scale and diversity, which may be beneficial after further training on the target dataset. For this reason, you will fine-tune the COCO model on cityscapes training set for semantic segmentation and analyze how its performance changes on the validation set (be consistent in the evaluation pipeline with previous step).
   In addition you should compare the two versions trained on Cityscapes (the one you trained and the one originally provided) on mIOU for the whole classes in the validation of cityscapes.

   General tips when fine-tuning the model, it is a great idea to use AMP (automatic mixed precision) to reduce training time. A good first experiment

to start out with is to finetune just the prediction head. Then you can gradually unfreeze the last layers and compare (check Colab time constraints).

Another technique that could be useful to be able to finetune without many resources is LoRA [9].

6. **Understand anomaly segmentation task and post-hoc methods**
   Get familiar with the task of anomaly segmentation, start by studying the problem and datasets [1,2] and then read up the most important anomaly segmentation post-hoc methods [7, 8].

7. **Pixel-based baselines**
   To develop the first baselines for anomaly segmentation let's start from a more conventional approach of using pixel-based approach and apply post-hoc methods using these models. In the drive folder you will find all the datasets you need to evaluate the model on (*Anomaly_Validation_Datasets.zip*).
   You can find the method for computing MaxLogit using a ERFNet pretrained model on the anomaly segmentation validation sets (the weights are also on github) in the Eval Folder.
   The post-hoc methods you need to run inference for are: MSP, Max Logit (already implemented), Max Entropy.
   Resources for max-entropy can be found here.

8. **Mask-based baselines**
   In this step you will evaluate the eomt model on the same anomaly segmentation datasets. In the repo you can find the eomt repository repository with all you will need for the project.
   Adapt the eval code that you used in the previous step to use an EoMT pretrained model. Keep in mind that we are switching into a Mask Architecture so the output of the model will be different.
   The post-hoc methods you need to run inference for are: MSP, Max Logit, Max Entropy and RbA, that can now be applied since you are using a mask Architecture.
   Resources for RbA can be found here.
   The baselines results should be reported in a table like this:

			SMIYC RA-21		SMIYC RO-21		FS L&F		FS Static		Road Anomaly	
Model	**mIoU**	**Method**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**
ERFNET	----	MSP										
		MaxLogit										
		Max Entropy										
EoMT	-----	MSP										
		MaxLogit										
		Max Entropy										
		RbA										

You should evaluate these results **for all the 3 checkpoints you have of EoMT:** the one trained on COCO, the one trained on cityscapes and your fine-tuned version (the mIOU will not change by changing the post-hoc method but it is different for different weights).

To get another baseline try Temperature scaling.

It is a method for confidence calibration for any classifier which could result in improving anomaly segmentation capabilities of a network. Hence, we use this technique while segmenting anomalies during inference. The goal of this part is to find the value of temperature that gives the best anomaly segmentation results.

PRO TIP: To make this fast, you can save the model predictions on the datasets and then try different temperatures with the saved logits. In that way you will need to run the model forward pass once.

		SMIYC RA-21		SMIYC RO-21		FS L&F		FS Static		Road Anomaly	
Method	**mIoU**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**	**AuPRC**	**FPR95**
MSP											
MSP(t = 0.5)											
MSP(t = 0.75)											
MSP(t = 1.1)											
MSP (best t)											

References:

1. SegmentMeIfYouCan: A Benchmark for Anomaly Segmentation
2. The Fishyscapes Benchmark Anomaly Detection for Semantic Segmentation
3. Per-Pixel Classification is Not All You Need for Semantic Segmentation, Bowen Cheng et. Al.
4. Mask2Former: Masked-attention Mask Transformer for Universal Image Segmentation
5. DINOv2: Learning Robust Visual Features without Supervision
6. Your ViT is Secretly an Image Segmentation Model (CVPR 2025)
7. RbA: Segmenting Unknown Regions Rejected by All
8. Scaling Out-of-Distribution Detection for Real-World Settings
9. LoRA: Low-Rank Adaptation of Large Language Models
10. ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation
11. Panoptic Segmentation (Kirillov et. Al.)
12. The Cityscapes Dataset for Semantic Urban Scene Understanding
13. Microsoft COCO: Common Objects in Context